

Wiederholung: Final

- **Was macht nochmal final?**
- final ist ein Modifikator, der eine Entität **unveränderlich** macht. Die genaue Bedeutung hängt davon ab, wo es verwendet wird:
- **final Klasse:** Von dieser Klasse kann nicht geerbt werden. Sie kann keine Kindklassen haben (z.B. final class ListUtil).
- **final Attribut/Variable:** Der Wert der Variable kann nach der ersten Zuweisung (oft im Konstruktor) nicht mehr geändert werden. Sie wird zu einer Konstante (z.B. private final int numberOfBeds; oder private final Map<CurrencyPair, ConversionRate> conversions).

Wiederholung zur Übung: 100 % Abdeckung

- **Warum 100% Abdeckung?**
- Die Aufgabenstellung verlangt 100% Testabdeckung (Test Coverage)
- Das Ziel von 100% Abdeckung ist sicherzustellen, dass **jede einzelne Zeile** deines Produktionscodes (also jede Zeile in ListUtil, PayDude, CurrencyConverter usw.) durch mindestens einen automatisierten Testfall ausgeführt wurde.
- Man will 100% Abdeckung, um "tote Winkel" in der Logik zu vermeiden. Wenn du nur den "Happy Path" (den Normalfall) testest, würdest du vielleicht 80% Abdeckung erreichen. Die restlichen 20% sind oft die Fehlerbehandlung (z.B. die if-Blöcke, die Exceptions werfen).
- **Beispiel PayDude:**
- `sumOfDepositedMoneyIsReturnedAsBalanced` testet den normalen deposit.
- Aber erst `depositNegativeAmountNotAllowed` testet den `if (amount < 0)`-Block und erreicht damit 100% Abdeckung für die `assertNotNegative`-Methode.

Wiederholung zur Übung: Custom Exception

- **Warum PayDudeException?**
- Dies ist eine **Eigene Exception** (Custom Exception)
- Man erstellt eine eigene Exception-Klasse (z.B. PayDudeException), um **spezifische Fehler** der eigenen Geschäftslogik klar zu benennen.
- Es ist für einen Entwickler (oder einen Test) viel aussagekräftiger, eine PayDudeException mit der Meldung "Go get some money, dude." zu sehen, als eine generische RuntimeException. Es macht den Code "sprechender" und die Fehlerursache sofort ersichtlich.

Wiederholung zur Übung: Enum

- **Was ist ein Enum?**
- Ein Enum (Aufzählung) ist ein spezieller Datentyp in Java, der eine **feste, vordefinierte Menge von Konstanten** repräsentiert.
- In der Übung CurrencyConverter wird enum Currency { EURO, DOLLAR, INVALID } verwendet.
- Das bedeutet, eine Variable vom Typ Currency kann *nur* einen dieser drei Werte annehmen. Es verhindert, dass jemand versucht, Währungen als freien Text (z.B. "Yen" oder "Dollar\$") zu übergeben, was zu Fehlern führen würde. Es schränkt die möglichen Eingaben auf eine bekannte, gültige Menge ein.

Wiederholung zur Übung: Of

- **Was macht das of?**
- Die of-Methode in `ConversionRate` ist eine **statische Factory-Methode**.
- Sie ist im Grunde eine "Bequemlichkeitsmethode", die den eigentlichen Konstruktor (`new ConversionRate(value)`) aufruft.
- Der Hauptgrund für ihre Verwendung ist die **Lesbarkeit**. In der Initialisierung der `CurrencyConverter`-Klasse liest sich:
 - `ConversionRate.of(1.06304)`
 - ...oft flüssiger und klarer als:
 - `new ConversionRate(1.06304)`
- Es ist eine gängige Konvention in modernem Java (besonders bei record-Typen), of-Methoden anstelle von öffentlichen Konstruktoren zu verwenden.

Wiederholung zur Übung: Exceptions

- **Was sind Exceptions?** Exceptions (Ausnahmen)
- Exceptions sind Fehlerereignisse, die während der Ausführung eines Programms auftreten und den normalen Ablauf unterbrechen. Sie signalisieren, dass ein unerwarteter oder fehlerhafter Zustand eingetreten ist.
- In deinen Übungen sind Beispiele:
- `NoSuchElementException` : Wird geworfen, wenn man das häufigste Element in einer *leeren* Liste sucht.
- `NegativeValueException` : Wird geworfen, wenn ein negativer Betrag konvertiert werden soll.
- `HospitalException`: Wird geworfen, wenn kein Patient mehr im Krankenhaus ist, aber `healNextPatient()` aufgerufen wird.

Wiederholung zur Übung: Throw/Catch

- **Was ist throw und catch?**
- throw (Werfen)
- throw ist das Schlüsselwort, das eine Exception *aktiv auslöst* (wirft). Man benutzt es, um dem Programm mitzuteilen, DASS der Fehler passiert ist.
- **Beispiel:**

```
if (amount < 0) {  
    throw new NegativeValueException("Negative amount cannot be converted.");  
}.
```
- catch (Fangen)
- catch ist das Schlüsselwort, das eine geworfene Exception *behandelt* (fängt). Es wird (meist in einem try-catch-Block) verwendet, um auf den Fehler zu reagieren und den Programmabsturz zu verhindern.
- In deinen Unit-Tests verwendest du eine spezielle Form des Fangens: `assertThrows`.
- **Beispiel:** `assertThrows(PayDudeException.class, () -> dude.deposit(-1));`.
- Dieser Befehl "fängt" die `PayDudeException` und prüft, ob sie wie erwartet geworfen wurde. Wenn *keine* Exception geworfen wird, schlägt der Test fehl.